

```

import numpy as np, pandas as pd, matplotlib.pyplot as plt
from tensorflow import keras

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train, x_test = x_train.reshape(-1, 784) / 255.0, x_test.reshape(-1, 784) / 255.0

opts = {"SGD": lambda lr: keras.optimizers.SGD(lr),
        "Momentum": lambda lr: keras.optimizers.SGD(lr, momentum=0.9),
        "RMSProp": lambda lr: keras.optimizers.RMSprop(lr),
        "Adam": lambda lr: keras.optimizers.Adam(lr)}
results, hist = [], {}

def run(name, layers=2, neurons=128, act="relu", opt="Adam", lr=0.001, bs=128, epochs=5, n=60000):
    keras.utils.set_random_seed(42)
    m = keras.Sequential([keras.Input(shape=(784,))] +
                        [keras.layers.Dense(neurons, activation=act) for _ in range(layers)] +
                        [keras.layers.Dense(10, activation="softmax")])
    m.compile(opts[opt](lr), "sparse_categorical_crossentropy", metrics=["accuracy"])
    h = m.fit(x_train[:n], y_train[:n], epochs=epochs, batch_size=bs, validation_split=0.1, verbose=0)
    loss, acc = m.evaluate(x_test, y_test, verbose=0)
    results.append(dict(run=name, layers=layers, neurons=neurons, act=act, opt=opt, lr=lr,
                        batch_size=bs, test_acc=round(acc, 4), test_loss=round(loss, 4)))
    hist[name] = h.history
    print(name, round(acc, 4))

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— **1s** 0us/step

```

run("single-layer", layers=0)
run("multi-layer")

for o in opts:
    # 4 optimizers
    run(f"opt={o}", opt=o, lr=0.01 if o in ("SGD", "Momentum") else 0.001)

run("Batch", opt="SGD", lr=0.1, n=5000, bs=5000, epochs=100)      # batch GD
run("Mini-batch", opt="SGD", lr=0.1, n=5000, bs=32, epochs=10)   # mini-batch GD
run("Stochastic", opt="SGD", lr=0.01, n=5000, bs=1, epochs=3)    # stochastic GD

for lr in [0.1, 0.01, 0.001]: run(f"lr={lr}", lr=lr)             # tuning
for bs in [32, 512]: run(f"bs={bs}", bs=bs)
for nn in [32, 256]: run(f"neurons={nn}", neurons=nn)
for a in ["tanh", "sigmoid"]: run(f"act={a}", act=a)

```

```

single-layer 0.9236
multi-layer 0.9713
opt=SGD 0.9135
opt=Momentum 0.9623
opt=RMSProp 0.9737
opt=Adam 0.9713
Batch 0.8866
Mini-batch 0.9294
Stochastic 0.9202
lr=0.1 0.4584
lr=0.01 0.9632
lr=0.001 0.9713
bs=32 0.9739
bs=512 0.9657
neurons=32 0.9581
neurons=256 0.9741
act=tanh 0.9714
act=sigmoid 0.9574

```

```
df = pd.DataFrame(results)
display(df)

def plot(keys, title):
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    for k in keys:
        ax[0].plot(hist[k]["val_accuracy"], label=k)
        ax[1].plot(hist[k]["val_loss"], label=k)
    ax[0].set_title(title + " - accuracy"); ax[1].set_title(title + " - loss")
    ax[0].legend(); plt.show()

plot([k for k in hist if k.startswith("opt=")], "Optimizers")
plot(["Batch", "Mini-batch", "Stochastic"], "Batch styles")
plot([k for k in hist if k.startswith("lr=")], "Learning rate")
```

